PRACTICAL NO. 01

28-11-2025

AIM: MLP for Classification.
Build a multi-layer perceptron for a classification problem. Experiment with different activation functions and layer configurations.

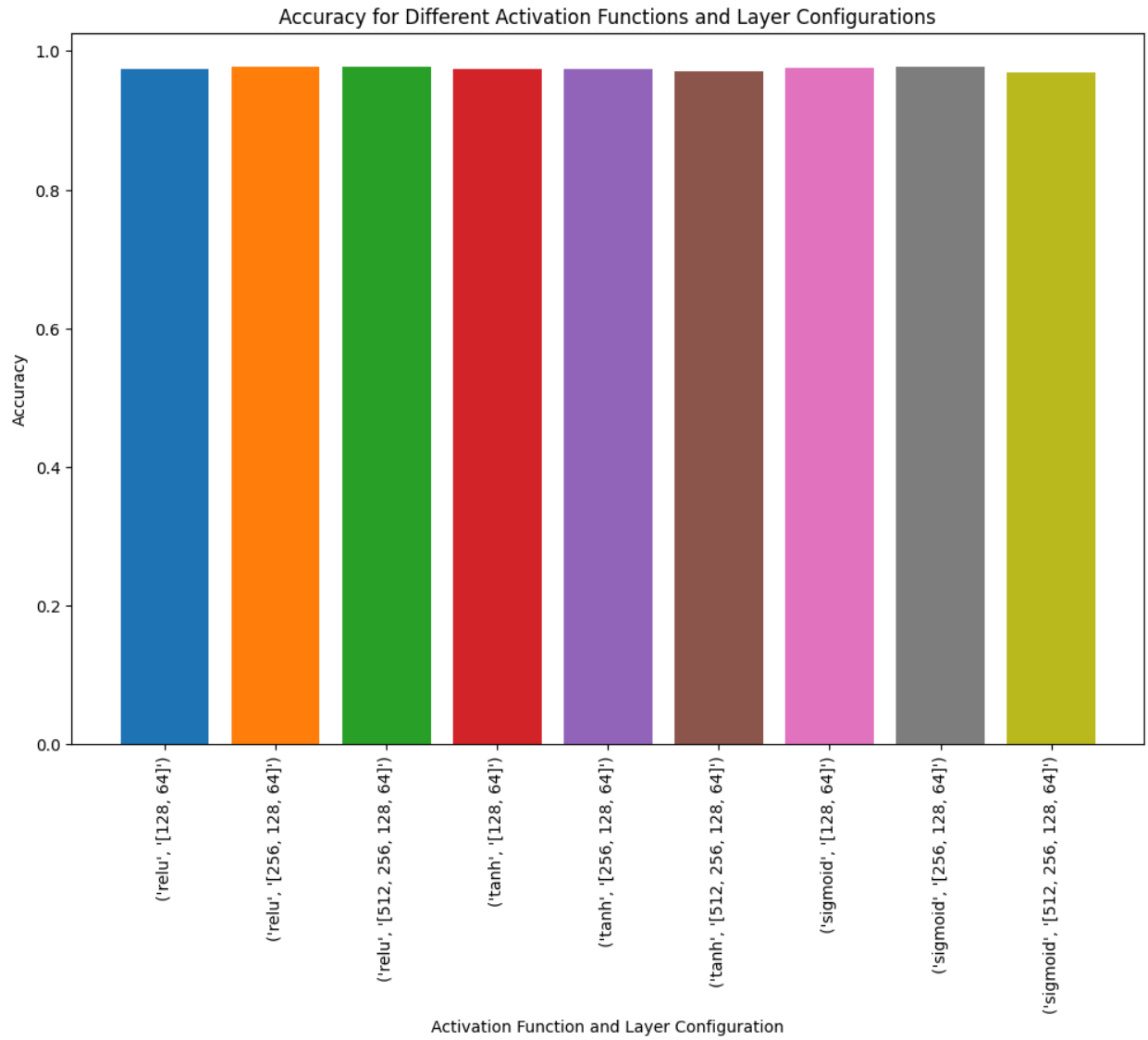
CODE & OUTPUT:

```
# On MNIST Dataset
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.optimizers import Adam
from sklearn.metrics import classification_report, confusion_matrix
```

```
# Load the MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize the dataset
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0

# One-hot encode the labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Build a Multi-Layer Perceptron model
def build_mlp(activation='relu', layer_config=[128, 64]):
    model = Sequential()
    model.add(Flatten(input_shape=(28, 28)))
    for units in layer_config:
        model.add(Dense(units, activation=activation))
    model.add(Dense(10, activation='softmax'))
                        model.compile(optimizer=Adam(learning_rate=0.001),
```

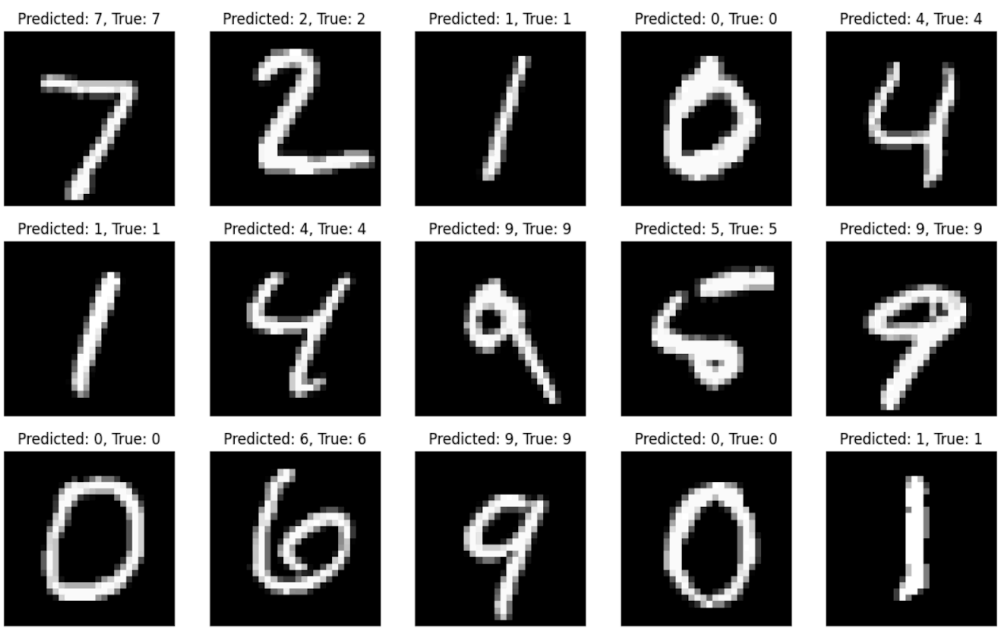
```
loss='categorical_crossentropy', metrics=['accuracy'])
    return model
```

```
# Experiment with different configurations
activations = ["relu", "tanh", "sigmoid"]
layer_configs = [[128, 64], [256, 128, 64], [512, 256, 128, 64]]

results = {}

for activation in activations:
    for config in layer_configs:
        model = build_mlp(activation=activation, layer_config=config)
         history = model.fit(X_train, y_train, epochs=10, batch_size=32,
validation_split=0.2, verbose=0)
        _, accuracy = model.evaluate(X_test, y_test, verbose=0)
        results[(activation, str(config))] = accuracy
             print(f'Activation: {activation}, Layer Config: {config},
Accuracy: {accuracy:.4f}')

# Visualize the results
plt.figure(figsize=(12, 8))
for key, value in results.items():
 plt.bar(str(key), value)
plt.xticks(rotation=90)
plt.xlabel('Activation Function and Layer Configuration')
plt.ylabel('Accuracy')
plt.title('Accuracy for Different Activation Functions and Layer
Configurations')
plt.show()
```




```
# Display classification report for the best model
best_model_key = max(results, key=results.get)
best_activation, best_config = best_model_key
best_model = build_mlp(activation=best_activation,
layer_config=eval(best_config))
best_model.fit(X_train, y_train, epochs=10, batch_size=32,
validation_split=0.2, verbose=0)
y_pred = np.argmax(best_model.predict(X_test), axis=1)
y_test_labels = np.argmax(y_test, axis=1)

print(f'Best Model: Activation: {best_activation}, Layer Config:
```

```
{best_config}')
print('Classification Report:')
print(classification_report(y_test_labels, y_pred))
print('Confusion Matrix:')
print(confusion_matrix(y_test_labels, y_pred))

# Display some test images with their predicted labels
plt.figure(figsize=(15, 15))
for i in range(25):
    plt.subplot(5, 5, i + 1)
    plt.imshow(X_test[i], cmap='gray')
    plt.title(f'Predicted: {y_pred[i]}, True: {y_test_labels[i]}')
    plt.axis('off')
plt.show(
```



PRACTICAL NO. 02

29-11-2025

AIM: CNN for Image Classification.
Create a CNN for Image Classification using a popular dataset MNIST or CIFAR-10.

CODE & OUTPUT:

```
# Import required libraries
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tensorflow import keras
from tensorflow.keras import layers, models
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.model_selection import train_test_split

# 1. Dataset Preparation
# ------------------------

# Load the MNIST dataset (28x28 grayscale images of handwritten digits)
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

# Normalize the image data to range [0, 1]
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Reshape to include channel dimension (28, 28, 1)
x_train = x_train[..., np.newaxis]
x_test = x_test[..., np.newaxis]

# Convert labels to one-hot encoded format
num_classes = 10
y_train_cat = keras.utils.to_categorical(y_train, num_classes)
y_test_cat = keras.utils.to_categorical(y_test, num_classes)

# Split training data into train and validation sets
x_train, x_val, y_train_cat, y_val_cat = train_test_split(
    x_train, y_train_cat, test_size=0.1, random_state=42
)
```

```
# 2. Define CNN Model Architecture
# ---------------------------------
model = models.Sequential()

# Convolutional Layer 1: Extract low-level features
model.add(layers.Conv2D(32,  (3,  3),  activation='relu',  input_shape=(28,
28, 1)))
model.add(layers.MaxPooling2D((2, 2))) # Reduce spatial dimensions

# Convolutional Layer 2
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

# Flatten the output and feed into Fully Connected Layers
model.add(layers.Flatten())
model.add(layers.Dense(128, activation='relu')) # Fully connected layer
model.add(layers.Dropout(0.5)) # Dropout for regularization

# Output layer with Softmax for multi-class classification
model.add(layers.Dense(num_classes, activation='softmax'))

# Display the model summary
model.summary()
```

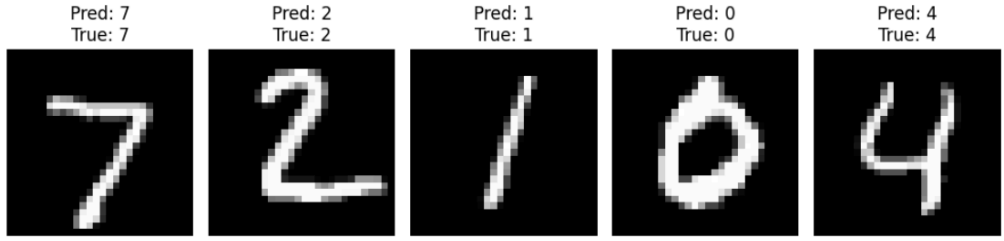
```
# 3. Compilation
# ---------------
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# 4. Training the Model
# ----------------------

# Define EarlyStopping to prevent overfitting
early_stop = keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=3, restore_best_weights=True
```

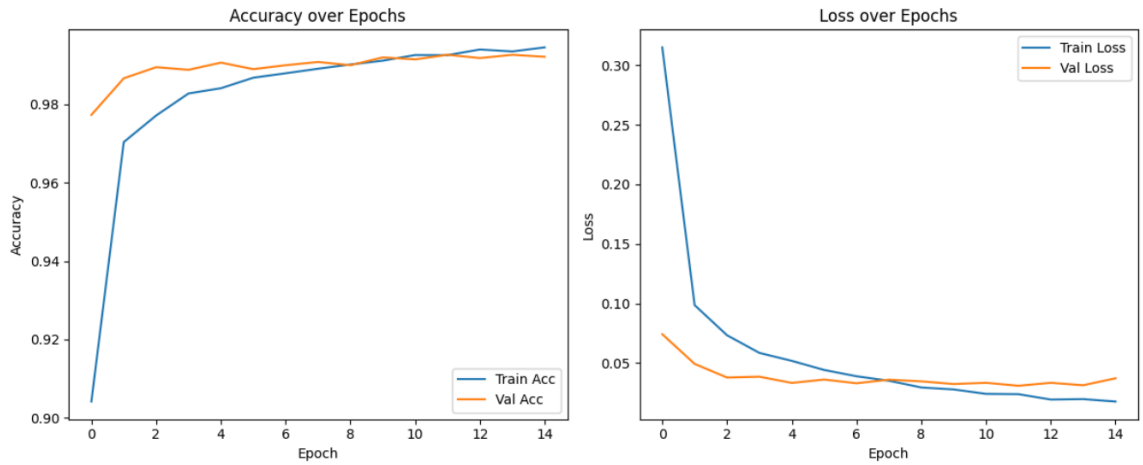
```
)

# Train the model
history = model.fit(
    x_train, y_train_cat,
    epochs=15,
    batch_size=128,
    validation_data=(x_val, y_val_cat),
    callbacks=[early_stop]
)
```

```
# 5. Testing and Evaluation
# --------------------------
test_loss, test_acc = model.evaluate(x_test, y_test_cat, verbose=2)
print(f"\nTest accuracy: {test_acc:.4f}, Test loss: {test_loss:.4f}")

# Predict on some test images
predictions = model.predict(x_test)
y_pred = np.argmax(predictions, axis=1)

# Display a few predictions
plt.figure(figsize=(10, 4))
for i in range(5):
    plt.subplot(1, 5, i+1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.title(f"Pred: {y_pred[i]}\nTrue: {y_test[i]}")
    plt.axis('off')
plt.tight_layout()
plt.show()

# 6. Performance Visualization
# -----------------------------

# Plot training & validation accuracy/loss
def plot_metrics(history):
    plt.figure(figsize=(12, 5))

    # Accuracy Plot
```

```
    plt.subplot(1, 2, 1)
    plt.plot(history.history['accuracy'], label='Train Acc')
    plt.plot(history.history['val_accuracy'], label='Val Acc')
    plt.title('Accuracy over Epochs')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.legend()

    # Loss Plot
    plt.subplot(1, 2, 2)
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Val Loss')
    plt.title('Loss over Epochs')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()

    plt.tight_layout()
    plt.show()

plot_metrics(history)

# Confusion Matrix (Visualized in the slides)
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8,6))
sns.heatmap(cm,         annot=True,         fmt='d',         cmap='Blues',
xticklabels=range(10), yticklabels=range(10))
plt.xlabel('Predicted')
plt.ylabel('Truth')
plt.title('Confusion Matrix')
plt.show()

# Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

```
313/313 - 3s - 9ms/step - accuracy: 0.9934 - loss: 0.0217

Test accuracy: 0.9934, Test loss: 0.0217
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step
```






```
# 7. Summary of Model Components
# -------------------------------
# - Conv2D Layers: Extract hierarchical features (edges, corners, shapes).
# - ReLU: Adds non-linearity to the feature maps.
# - MaxPooling2D: Reduces spatial resolution while keeping prominent features.
# - Dense Layers: Learn non-linear decision boundaries.
# - Softmax: Outputs class probabilities for multi-class classification
```

PRACTICAL NO. 03

08-12-2025

AIM: RNN Implementation.
[Stock Price Prediction using RNN (LSTM and GRU).
To implement Recurrent Neural Network models (LSTM and GRU) for stock price prediction and compare their performance using historical stock data.]

CODE:

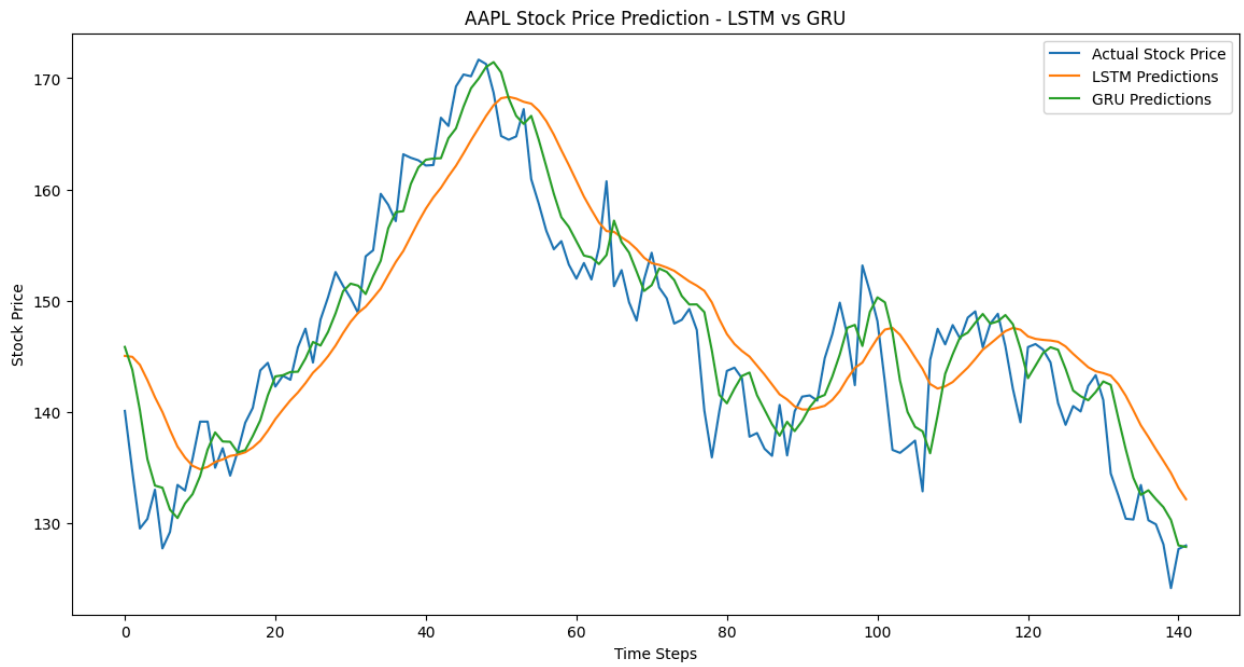
```
# Import necessary libraries
import yfinance as yf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, GRU, Dense

# Load stock data from Yahoo Finance (e.g., Apple stock)
stock_symbol = "AAPL"
df = yf.download(stock_symbol, start="2019-01-01", end="2023-01-01")

# Use the 'Close' price for modeling
data = df[['Close']].values

# Normalize the data
scaler = MinMaxScaler(feature_range=(0, 1))
data_scaled = scaler.fit_transform(data)

# Split data into train and test sets
train_size = int(len(data_scaled) * 0.8)
train, test = data_scaled[0:train_size], data_scaled[train_size:]

# Convert the data into time series format
def create_dataset(dataset, look_back=60):
    X, y = [], []
    for i in range(len(dataset) - look_back):
        X.append(dataset[i:i + look_back, 0])
        y.append(dataset[i + look_back, 0])
```

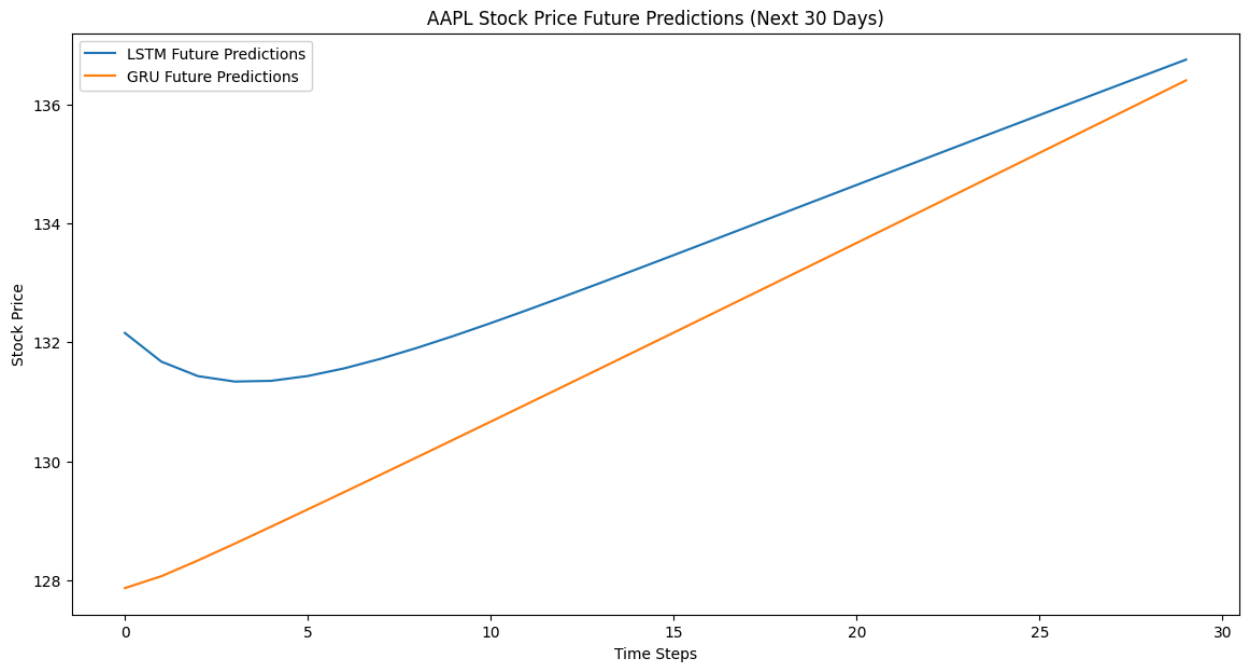
```
    return np.array(X), np.array(y)

look_back = 60
X_train, y_train = create_dataset(train, look_back)
X_test, y_test = create_dataset(test, look_back)

# Reshape input to be [samples, time steps, features]
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

# Build LSTM model
model_lstm = Sequential()
model_lstm.add(LSTM(50, input_shape=(look_back, 1)))
model_lstm.add(Dense(1))
model_lstm.compile(loss='mean_squared_error', optimizer='adam')

# Build GRU model
model_gru = Sequential()
model_gru.add(GRU(50, input_shape=(look_back, 1)))
model_gru.add(Dense(1))
model_gru.compile(loss='mean_squared_error', optimizer='adam')

# Train both models
model_lstm.fit(X_train, y_train, epochs=20, batch_size=32, verbose=1)
model_gru.fit(X_train, y_train, epochs=20, batch_size=32, verbose=1)

# Make predictions
lstm_predictions = model_lstm.predict(X_test)
gru_predictions = model_gru.predict(X_test)

# Inverse transform predictions and actual values
lstm_predictions = scaler.inverse_transform(lstm_predictions)
gru_predictions = scaler.inverse_transform(gru_predictions)
y_test_actual = scaler.inverse_transform(y_test.reshape(-1, 1))

# Plot the predictions and actual values
plt.figure(figsize=(14, 7))
plt.plot(y_test_actual, label='Actual Stock Price')
plt.plot(lstm_predictions, label='LSTM Predictions')
```

```
plt.plot(gru_predictions, label='GRU Predictions')
plt.title(f'{stock_symbol} Stock Price Prediction - LSTM vs GRU')
plt.xlabel('Time Steps')
plt.ylabel('Stock Price')
plt.legend()
plt.show()

# Future predictions (next 30 days)
future_steps = 30
last_sequence = X_test[-1] # Last sequence in the test data

# Predict future prices using the LSTM model
lstm_future_predictions = []
for _ in range(future_steps):
     prediction = model_lstm.predict(last_sequence.reshape(1, look_back,
1))
    lstm_future_predictions.append(prediction[0][0])
    last_sequence = np.append(last_sequence[1:], prediction[0])

# Predict future prices using the GRU model
last_sequence = X_test[-1] # Reset sequence
gru_future_predictions = []
for _ in range(future_steps):
     prediction = model_gru.predict(last_sequence.reshape(1, look_back,
1))
    gru_future_predictions.append(prediction[0][0])
    last_sequence = np.append(last_sequence[1:], prediction[0])

# Inverse transform future predictions
lstm_future_predictions                                                =
scaler.inverse_transform(np.array(lstm_future_predictions).reshape(-1,
1))
gru_future_predictions                                                 =
scaler.inverse_transform(np.array(gru_future_predictions).reshape(-1,
1))

# Plot future predictions
plt.figure(figsize=(14, 7))
plt.plot(lstm_future_predictions, label='LSTM Future Predictions')
```

```
plt.plot(gru_future_predictions, label='GRU Future Predictions')
plt.title(f'{stock_symbol} Stock Price Future Predictions (Next 30 Days)')
plt.xlabel('Time Steps')
plt.ylabel('Stock Price')
plt.legend()
plt.show()
```

OUTPUT:




AAPL Stock Price Future Predictions (Next 30 Days)
LSTM Future Predictions
GRU Future Predictions
Stock Price
Time Steps
136
134
132
130
128
0
5
10
15
20
25
30

PRACTICAL NO. 04

00-00-2025

AIM: Autoencoder for Dimensionality Reduction.
[Autoencoder for Image Reconstruction and Latent Space Visualization.
To implement a basic autoencoder on the MNIST dataset for image compression and reconstruction, and to visualize the learned latent space representation using t-SNE.]

CODE:

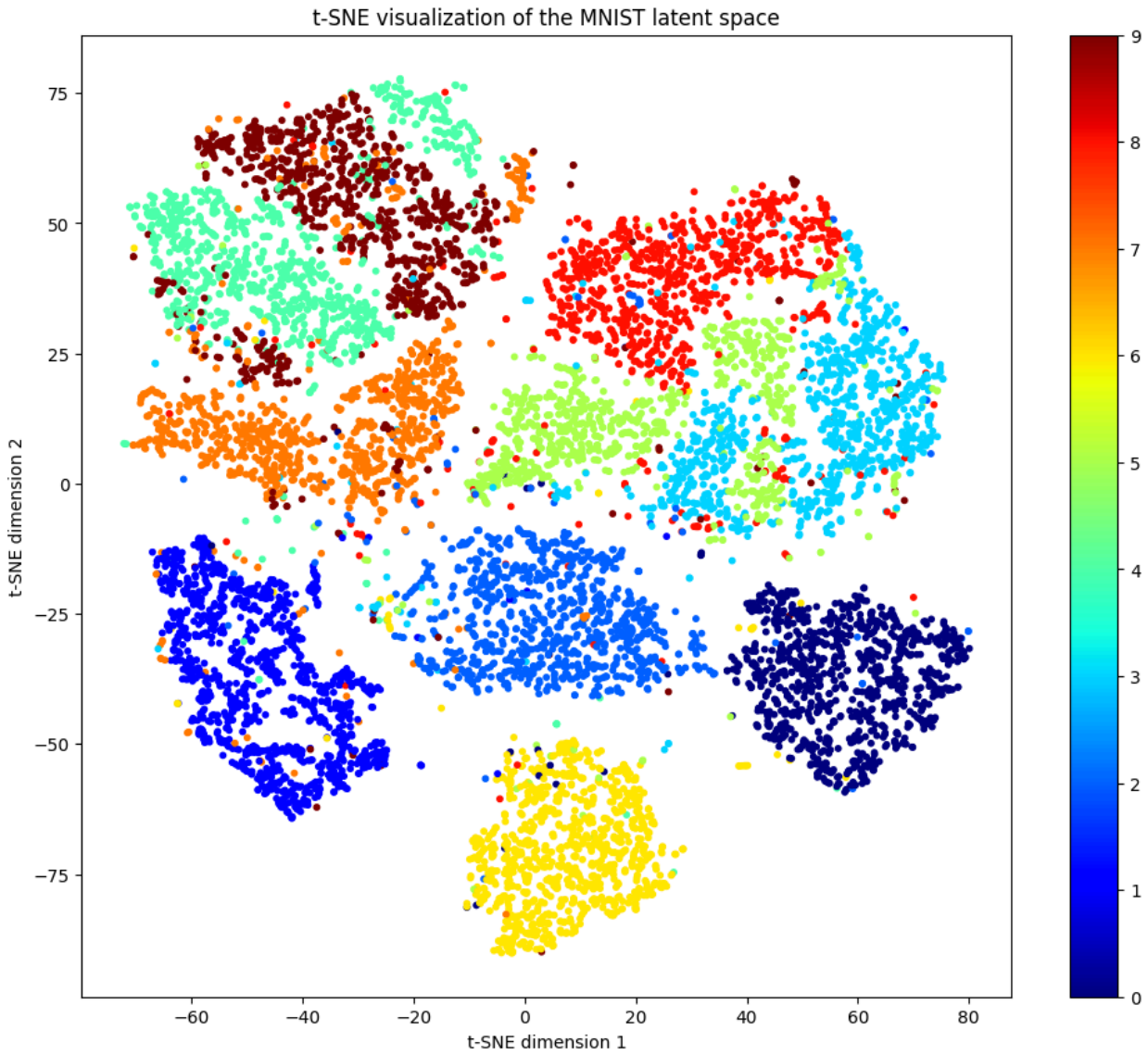
```
import numpy as np
import matplotlib.pyplot as plt
from keras.datasets import mnist
from keras.models import Model
from keras.layers import Input, Dense
from keras.optimizers import Adam
from sklearn.manifold import TSNE

# -------------------------------------
# 1. Load and Preprocess the MNIST Data
# -------------------------------------

(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize pixel values to be between 0 and 1
x_train = x_train.astype('float32') / 255
x_test = x_test.astype('float32') / 255.

# Flatten the 28x28 images into vectors of 784 elements
x_train = x_train.reshape((len(x_train), np.prod(x_train.shape[1:])))
x_test = x_test.reshape((len(x_test), np.prod(x_test.shape[1:])))


# ------------------------------
# 2. Build the autoencoder model
# ------------------------------

# This is the size of our encoded representations
encoding_dim = 32 # 32 floats -> compression of factor 24.5, assuming
the input is 784 floats
```

```
# This is our input placeholder
input_img = Input(shape=(784,))

# "encoded" is the encoded representation of the input
encoded = Dense(encoding_dim, activation='relu')(input_img)

# "decoded " is the Lossy reconstruction of the input
decoded = Dense(784, activation='sigmoid')(encoded)

# This model maps an input to its reconstruction
autoencoder = Model(input_img, decoded)

# This model maps an input to its encoded representation
encoder = Model(input_img, encoded)

# Create a placeholder for an encoded (32-dimensional) input
encoded_input = Input(shape=(encoding_dim,))

# Retrieve the Last Layer of the autoencoder model
decoder_layer = autoencoder.layers[-1]

# Create the decoder model
decoder = Model(encoded_input, decoder_layer(encoded_input))

# Compile the autoencoder
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')


# ------------------------
# 3. Train the autoencoder
# ------------------------
history = autoencoder.fit(x_train, x_train, epochs=50, batch_size=256,
shuffle=True, validation_data=(x_test, x_test))


# -------------------------------------------------------
# 4. Predict on the test data to get reconstructed images
# -------------------------------------------------------
```

```python
decoded_imgs = autoencoder.predict(x_test)


# ---------------------------------------------------
# 5. Visualize the original and reconstructed images
# ---------------------------------------------------

n = 10 # How many digits we will display
plt.figure(figsize=(20, 4))
for i in range(n):
    # Display original
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    if i == 0:
        ax.set_title('Original Images', loc='left')

    # display reconstruction
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    if i == 0:
        ax.set_title('Reconstructed Images', loc='left')
plt.show()


# --------------------------------------------
# 6. Plotting the Training and Validation Loss
# --------------------------------------------
plt.figure(figsize=(10, 5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss During Training')
plt.xlabel('Epoch')
plt.ylabel('Loss')
```

```
plt.legend()
plt.grid(True)
plt.show()


# ----------------------------------------
# 7. Visualize the Latent Space using t-SNE
# ----------------------------------------

# Use the encoder to get the latent representations of the test data
encoded_imgs = encoder.predict(x_test)

# Use t-SNE to reduce the dimensionality of the latent space to 2D
tsne = TSNE(n_components=2, random_state=42)
encoded_imgs_2d = tsne.fit_transform(encoded_imgs)

# Plot the 2D Latent space
plt.figure(figsize=(12, 10))
plt.scatter(encoded_imgs_2d[:,      0],      encoded_imgs_2d[:,      1],
c=y_test,cmap='jet',s=10)
plt.colorbar()
plt.title('t-SNE visualization of the MNIST latent space')
plt.xlabel('t-SNE dimension 1')
plt.ylabel('t-SNE dimension 2')
plt.show()
```

OUTPUT:


t-SNE visualization of the MNIST latent space
t-SNE dimension 1
t-SNE dimension 2
−60
−40
−20
0
20
40
60
80
75
50
25
0
−25
−50
−75
9
8
7
6
5
4
3
2
1
0

PRACTICAL NO. 05

00-00-2025

AIM: Gradient-Based Optimization.
[Gradient Descent Optimization using TensorFlow.]

Problem Statement:
Optimization is a critical step in machine learning where the goal is to minimize a loss function by iteratively adjusting model parameters. In this lab, w work on minimizing a simple quadratic function f(x) = (x - 3) ^ 2 which has its minimum at x = 3.
The task is divided into the following parts:

1. Manual Gradient Descent:
   - ❖ Compute gradients of the loss function using TensorFlow's Gradient Tape.
   - ❖ Update the parameter x manually using the gradient descent formula.
   - ❖ Visualize the reduction in loss over optimization steps.
2. Using TensorFlow Optimizers:
   - ❖ Implement the same optimization task using TensorFlow's built-in Adam optimizer.
   - ❖ Compare the behavior and performance of manual gradient descent with the Adam optimizer.
3. Visualization and Analysis:
   Plot the loss values against optimization steps for both manual and optimizer-based approaches. Analyze the convergence behavior and discuss the differences between t two methods.

CODE:

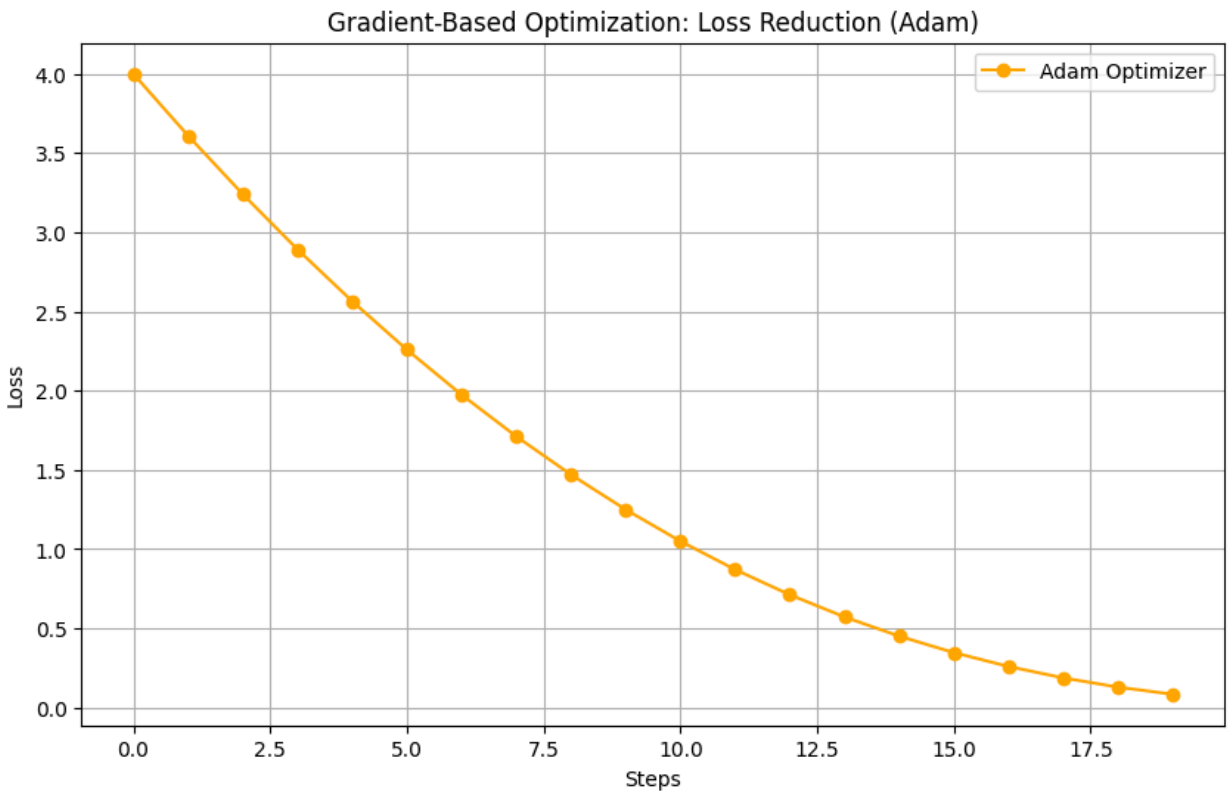
```
import tensorflow as tf
import matplotlib.pyplot as plt



# ------------------------------------------------------------
# PART 1: Function Definition
# ------------------------------------------------------------

# Define a simple quadratic loss function: f(x) = (x - 3) ^ 2
# The minimum of this function is at x = 3
def loss_function(x):
  return (x-3) **2
```

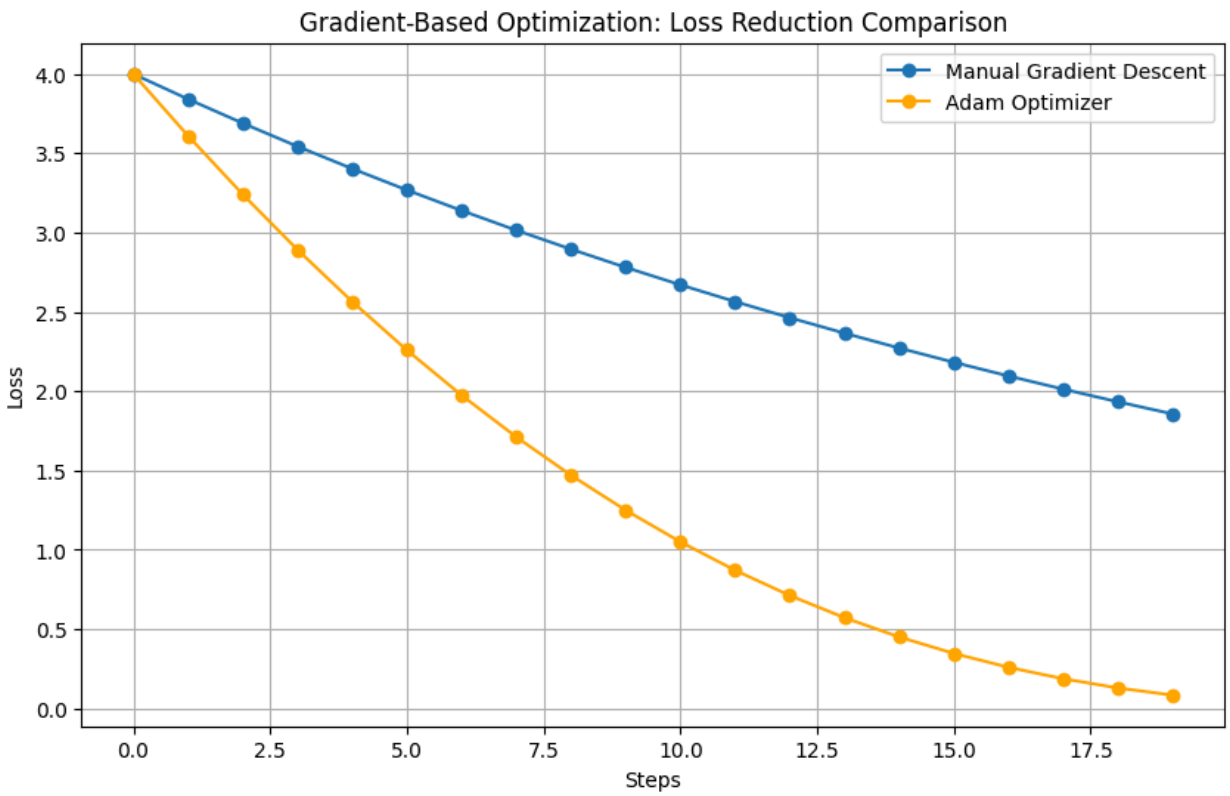
```
# ------------------------------
# PART 2: Manual Gradient Descent
# ------------------------------

# Initialize a variable (trainable parameter) with an arbitrary value
x_manual = tf.Variable(initial_value=5.0, trainable=True, dtype
=tf.float32)

# Learning rate for gradient-based optimization
learning_rate = 0.01

# Lists to store values for visualization
steps = []
loss_values=[]

# Gradient descent optimization for 20 steps
for step in range(20):
  with tf.GradientTape() as tape:
    # Compute the Loss
    loss = loss_function(x_manual)
   # Compute the gradient of the loss with respect to the variable
x_manual
  gradients = tape.gradient(loss, [x_manual])

  # Update the variable using the gradient
  x_manual.assign_sub(learning_rate * gradients[0]) #x_manual = x_manual
- Learning_rate * gradient

  # Store the step and Loss value for visualization
  steps.append(step)
  loss_values.append(loss.numpy())
  # Print the details of each step
    print(f"Step {step + 1}: x = {x_manual.numpy():.4f}, Loss =
{loss.numpy():.4f}")


# ------------------------------------------------
```

```
# PART 3: Visualization for Manual Gradient Descent
# -------------------------------------------------

# Plot the Loss values over the optimization steps
plt.figure(figsize=(10, 6))
plt.plot(steps,   loss_values,   marker='o',   label='Manual   Gradient
Descent')
plt.title('Gradient-Based Optimization: Loss Reduction (Manual)')
plt.xlabel('Steps')
plt.ylabel('Loss')
plt.grid(True)
plt.legend()
plt.show()


# -----------------------------------
# PART 4: Using TensorFlow Optimizers
# -----------------------------------

#  Reset  the  variable  x  for  optimization  using  TensorFlow's  Adam
optimizer
x_adam    =    tf.Variable(initial_value    =5.0,    trainable    =True,
dtype=tf.float32)
# Define an Adam optimizer
optimizer = tf.keras.optimizers.Adam(learning_rate=0.1)
# Lists to store values for visualization
adam_steps=[]
adam_loss_values =[]
# Optimization using Adam for 20 steps
for step in range(20):
  with tf.GradientTape() as tape:
    # Compute the Loss
    loss = loss_function(x_adam)
  # Compute the gradients
  gradients = tape.gradient(loss, [x_adam])
  # Apply the gradients using the optimizer
  optimizer.apply_gradients(zip(gradients, [x_adam]))
  # Store the step and Loss value for visualization
  adam_steps.append(step)
```

```
  adam_loss_values.append(loss.numpy())
  # Print the details of each step
   print(f"Step {step + 1} (Adam): x = {x_adam.numpy():.4f}, Loss = {loss.numpy():.4f}")

# Plot the Loss values for Adam optimization
plt.figure(figsize=(10, 6))
plt.plot(adam_steps, adam_loss_values, marker='o', color='orange', label='Adam Optimizer')
plt.title('Gradient-Based Optimization: Loss Reduction (Adam)')
plt.xlabel('Steps')
plt.ylabel('Loss')
plt.grid(True)
plt.legend()
plt.show()


# ------------------------------
# PART 5: Combined Visualization
# ------------------------------

plt.figure(figsize=(10, 6))
plt.plot(steps, loss_values, marker='o', label='Manual Gradient Descent')
plt.plot(adam_steps, adam_loss_values, marker='o', color='orange', label='Adam Optimizer')
plt.title('Gradient-Based Optimization: Loss Reduction Comparison')
plt.xlabel('Steps')
plt.ylabel('Loss')
plt.grid(True)
plt.legend()
plt.show()
```

OUTPUT:


Gradient-Based Optimization: Loss Reduction (Adam)
Adam Optimizer
Loss
Steps
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5


Gradient-Based Optimization: Loss Reduction Comparison
Manual Gradient Descent
Adam Optimizer
Loss
Steps
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5

PRACTICAL NO. 06

[00-00-2025]

AIM: Implementation of GAN.
[Generative Adversarial Network (GAN) on MNIST.]

CODE:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt


# -----------------------------
# 1. SETUP AND DATA PREPARATION
# -----------------------------

# Check if TensorFlow is able to detect a GPU
print(tf.config.list_physical_devices('GPU'))

# Set the GPU device to use
device_name = '/device:GPU:0'

# Load MNIST dataset
mnist = tf.keras.datasets.mnist
(train_images, train_labels), (_, _) = mnist.load_data()

# Normalize the images to [-1, 1]
train_images = (train_images.astype('float32') - 127.5) / 127.5

# Reshape the images to (28, 28, 1) and add a channel dimension
train_images = np.expand_dims(train_images, axis=-1)

# Batch and shuffle the data
BUFFER_SIZE = 60000
BATCH_SIZE = 256
train_dataset                                                          =
tf.data.Dataset.from_tensor_slices(train_images).shuffle(BUFFER_SIZE).ba
tch(BATCH_SIZE)
```

```
# ----------------------
# 2. MODEL ARCHITECTURES
# ----------------------

def make_generator_model():
    model = tf.keras.Sequential()
    # Initial dense layer
              model.add(tf.keras.layers.Dense(7*7*256,    use_bias=False,
input_shape=(100,)))
    model.add(tf.keras.layers.BatchNormalization())
    model.add(tf.keras.layers.LeakyReLU())

    # Reshape to start convolutional process
    model.add(tf.keras.layers.Reshape((7, 7, 256)))
    assert model.output_shape == (None, 7, 7, 256)

    # First Transposed Convolution (keeps 7x7 size)
     model.add(tf.keras.layers.Conv2DTranspose(128,  (5,  5),  strides=(1,
1), padding='same', use_bias=False))
    assert model.output_shape == (None, 7, 7, 128)
    model.add(tf.keras.layers.BatchNormalization())
    model.add(tf.keras.layers.LeakyReLU())

    # Second Transposed Convolution (upsamples to 14x14)
      model.add(tf.keras.layers.Conv2DTranspose(64,  (5,  5),  strides=(2,
2), padding='same', use_bias=False))
    assert model.output_shape == (None, 14, 14, 64)
    model.add(tf.keras.layers.BatchNormalization())
    model.add(tf.keras.layers.LeakyReLU())

    # Final Transposed Convolution (upsamples to 28x28 grayscale image)
     model.add(tf.keras.layers.Conv2DTranspose(1, (5, 5), strides=(2, 2),
padding='same', use_bias=False, activation='tanh'))
    assert model.output_shape == (None, 28, 28, 1)

    return model
```

```
def make_discriminator_model():
    model = tf.keras.Sequential()
    # First convolutional layer
        model.add(tf.keras.layers.Conv2D(64,  (5,  5),  strides=(2,  2),
padding='same', input_shape=[28, 28, 1]))
    model.add(tf.keras.layers.LeakyReLU())
    model.add(tf.keras.layers.Dropout(0.3))

    # Second convolutional layer
        model.add(tf.keras.layers.Conv2D(128,  (5,  5),  strides=(2,  2),
padding='same'))
    model.add(tf.keras.layers.LeakyReLU())
    model.add(tf.keras.layers.Dropout(0.3))

    # Classifier head
    model.add(tf.keras.layers.Flatten())
    model.add(tf.keras.layers.Dense(1))

    return model


# --------------------------------
# 3. LOSS FUNCTIONS AND OPTIMIZERS
# --------------------------------

cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)

def discriminator_loss(real_output, fake_output):
    # Loss for correctly identifying real and fake images
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    total_loss = real_loss + fake_loss
    return total_loss

def generator_loss(fake_output):
    # Loss for how well the generator fooled the discriminator
    return cross_entropy(tf.ones_like(fake_output), fake_output)

# Instantiate models
```

```
generator = make_generator_model()
discriminator = make_discriminator_model()

# Define Adam optimizers
generator_optimizer = tf.keras.optimizers.Adam(1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)

# --- 4. TRAINING LOOP ---
EPOCHS = 100
noise_dim = 100
num_examples_to_generate = 16

@tf.function
def train_step(images):
    # Generate random noise seed
    noise = tf.random.normal([BATCH_SIZE, noise_dim])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        # Generate fake images
        generated_images = generator(noise, training=True)

        # Get discriminator evaluations
        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        # Calculate losses
        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

    # Compute and apply gradients
                gradients_of_generator    =    gen_tape.gradient(gen_loss,
generator.trainable_variables)
            gradients_of_discriminator   =   disc_tape.gradient(disc_loss,
discriminator.trainable_variables)

          generator_optimizer.apply_gradients(zip(gradients_of_generator,
generator.trainable_variables))

discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator,
```

```
discriminator.trainable_variables))


# ---------------------------------
# 5. IMAGE GENERATION AND EXECUTION
# ---------------------------------

def generate_and_save_images(model, epoch, test_input):
    # Predictions and rescale from [-1, 1] to [0, 1] for plotting
    predictions = model(test_input, training=False)
    predictions = (predictions + 1) / 2.0

    # Plot results in a 4x4 grid
    fig = plt.figure(figsize=(4, 4))
    for i in range(predictions.shape[0]):
        plt.subplot(4, 4, i+1)
        plt.imshow(predictions[i, :, :, 0], cmap='gray')
        plt.axis('off')

    plt.savefig('image_at_epoch_{:04d}.png'.format(epoch))
    plt.show()

# Fixed noise for consistent evaluation
fixed_noise = tf.random.normal([num_examples_to_generate, noise_dim])

# Main training execution
for epoch in range(EPOCHS):
    for image_batch in train_dataset:
        train_step(image_batch)

    # Save images every 10 epochs
    if (epoch + 1) % 10 == 0:
        generate_and_save_images(generator, epoch + 1, fixed_noise)

    print('Epoch {} completed'.format(epoch + 1))
```

OUTPUT:

PRACTICAL NO. 07

01-02-2026

AIM: Comparing different Optimizers.
[Comparison of Optimizers using PyTorch on Two Moons Dataset.
To implement and compare different optimization algorithms in PyTorch for binary classification using a simple neural network on the Two Moons dataset.]

CODE:

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader

# -----------------------------------
# 0) Reproducibility
# -----------------------------------

seed = 7
np.random.seed(seed)
torch.manual_seed(seed)

# -----------------------------------
# 1) Data: Two Moons (binary classification)
# -----------------------------------

X, y = make_moons(n_samples=1200, noise=0.18, random_state=seed)
X = StandardScaler().fit_transform(X)

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=seed, stratify=y
```

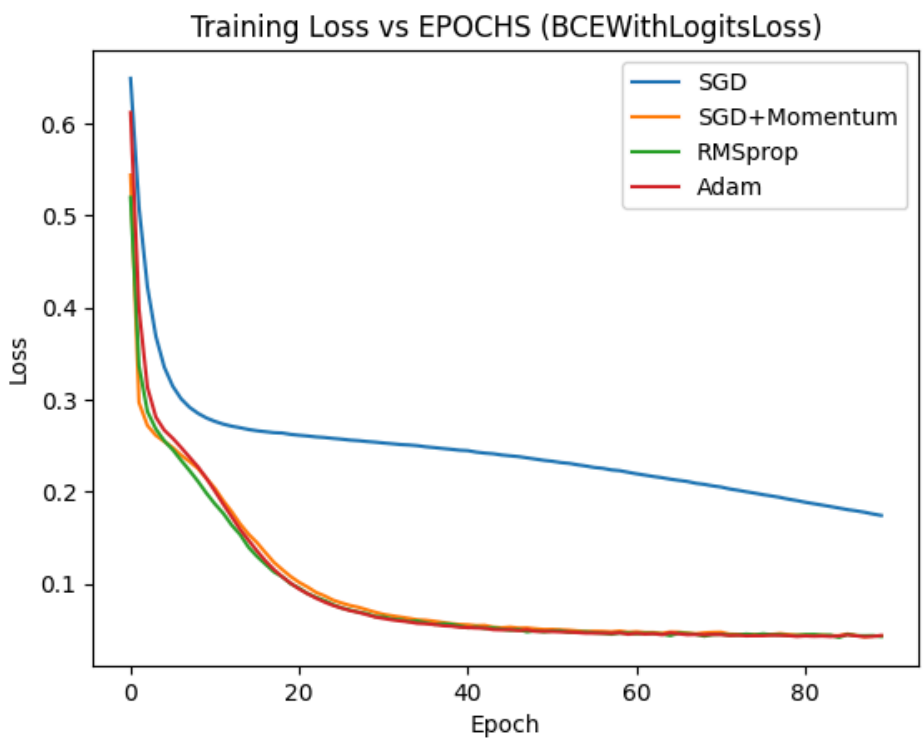
```
)

X_train_t = torch.tensor(X_train, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.float32).view(-1,1)
X_val_t = torch.tensor(X_val, dtype=torch.float32)
y_val_t = torch.tensor(y_val, dtype=torch.float32).view(-1,1)

train_loader = DataLoader(
    TensorDataset(X_train_t, y_train_t),
    batch_size=64, shuffle=True
)

criterion = nn.BCEWithLogitsLoss() # sigmoid is built-in via Logits loss

# ------------------------------------
# 2) Optimizers from library (torch.optim)
# ------------------------------------

optimizer_builders = {
    "SGD":          lambda params: optim.SGD(params, lr=0.08),
          "SGD+Momentum":  lambda  params:  optim.SGD(params,  lr=0.06,
momentum=0.9),
      "RMSprop":          lambda  params:  optim.RMSprop(params,  lr=0.01,
alpha=0.9),
    "Adam":         lambda params: optim.Adam(params, lr=0.01),
}

EPOCHS = 90

histories ={} # name -> dict of lists
final_metrics = {} # name -> (train_loss, val_loss, val_acc)

# ------------------------------------
# 3) Train the same model separately with each optimizer
# ------------------------------------

for name, make_opt in optimizer_builders.items():
  # Same init for fair competition
  torch.manual_seed(seed)
```

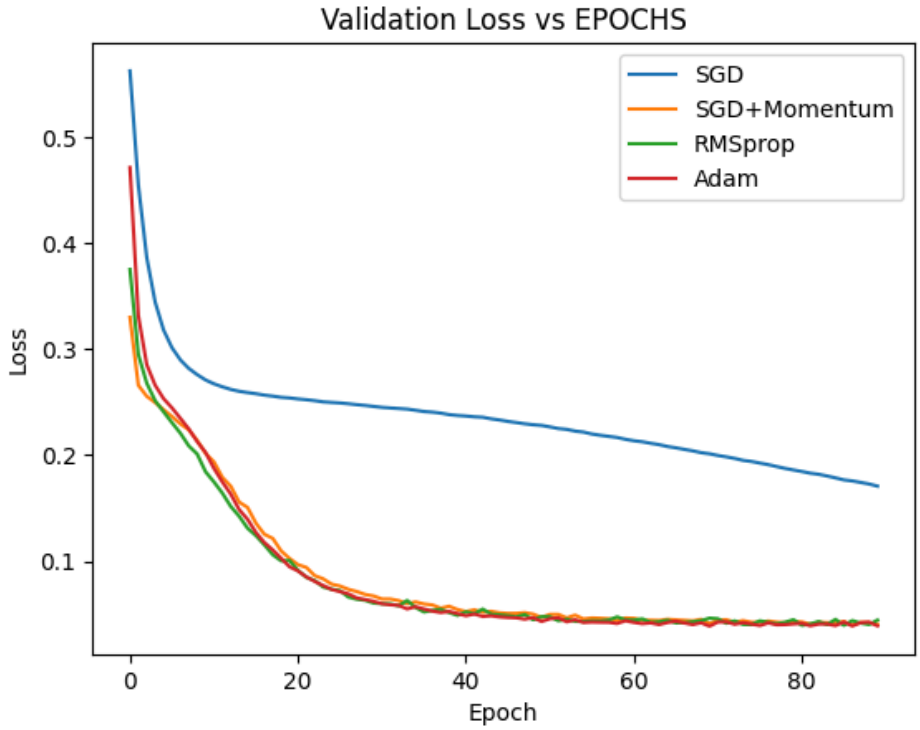
```python
model = nn.Sequential(
    nn.Linear(2, 16),
    nn.ReLU(),
    nn.Linear(16, 1) # logits
)

optimizer = make_opt(model.parameters())

train_loss_list = []
val_loss_list = []
val_acc_list = []

for epoch in range(EPOCHS):
  model.train()
  running_loss = 0.0

  for xb, yb in train_loader:
    optimizer.zero_grad(set_to_none=True)
    logits = model(xb)
    loss = criterion(logits, yb)
    loss.backward()
    optimizer.step()
    running_loss += loss.item() * xb.size(0)

  train_loss = running_loss / len(train_loader.dataset)

  # Validation (no function; inline)
  model.eval()
  with torch.no_grad():
    val_logits = model(X_val_t)
    val_loss = criterion(val_logits, y_val_t).item()
    val_probs = torch.sigmoid(val_logits)
    val_preds = (val_probs >= 0.5).float()
    val_acc = (val_preds.eq(y_val_t)).float().mean().item()

  train_loss_list.append(train_loss)
  val_loss_list.append(val_loss)
  val_acc_list.append(val_acc)
```

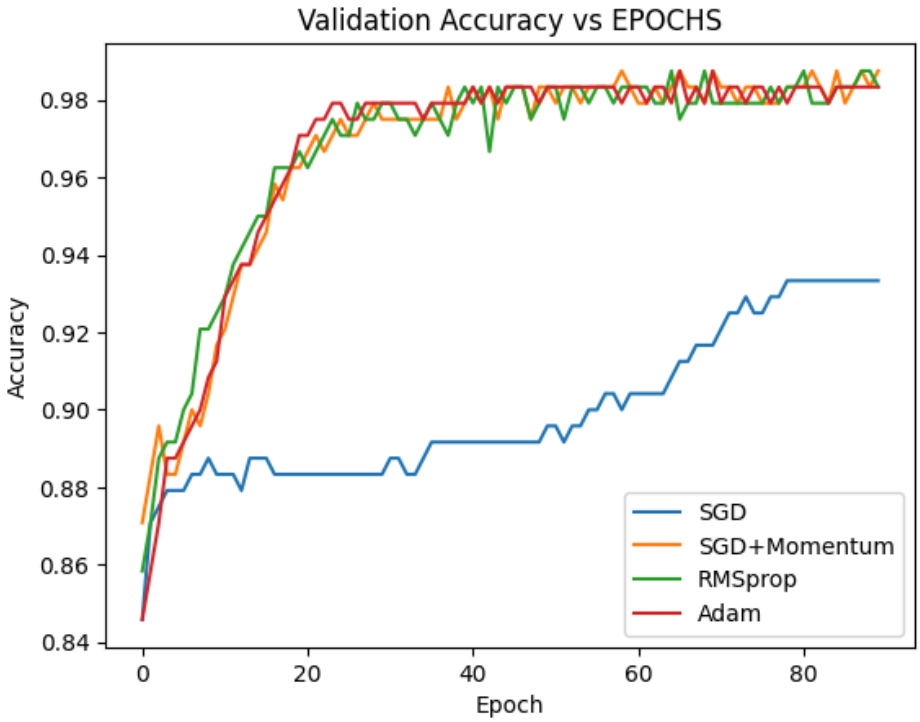
```
  histories[name] = {
      "train_loss":  train_loss_list,
      "val_loss": val_loss_list,
      "val_acc": val_acc_list
    }

     final_metrics[name]   =   (train_loss_list[-1],   val_loss_list[-1],
val_acc_list[-1])

# ------------------------------------
# 4) Plots
# ------------------------------------

plt.figure()
for name in histories:
  plt.plot(histories[name]["train_loss"], label=name)
plt.title("Training Loss vs EPOCHS (BCEWithLogitsLoss)")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()

plt.figure()
for name in histories:
  plt.plot(histories[name]["val_loss"], label=name)
plt.title("Validation Loss vs EPOCHS")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()

plt.figure()
for name in histories:
  plt.plot(histories[name]["val_acc"], label=name)
plt.title("Validation Accuracy vs EPOCHS")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
```

```
plt.show()

# ------------------------------------
# 5) Print final summary table
# ------------------------------------

print("Final Metrics after training: ")
print("Optimizer\tTrain Loss\tVal Loss\tVal Acc")
for name, (t1, v1, va) in final_metrics.items():
  print(f"{name:13s} {t1:8.4f} {v1:8.4f} {va*100:6.2f}")
```

OUTPUT:




Validation Loss vs EPOCHS
SGD
SGD+Momentum
RMSprop
Adam
Loss
0.5
0.4
0.3
0.2
0.1
0
20
40
60
80
Epoch


Validation Accuracy vs EPOCHS
Accuracy
0.98
0.96
0.94
0.92
0.90
0.88
0.86
0.84
0
20
40
60
80
Epoch
SGD
SGD+Momentum
RMSprop
Adam

PRACTICAL NO. 08

07-02-2026

AIM: Segmentation & Object Detection using FCN FRCNN / Segmentation: R-CNN and Masked R-CNN

CODE:

This code is for Google Colab

```
import torch
import torchvision
from torchvision import transforms
import matplotlib.pyplot as plt
from PIL import Image

# Load pre-trained models for segmentation and detection
fcn_model =
torchvision.models.segmentation.fcn_resnet50(pretrained=True)
fcn_model.eval()
faster_rcnn_model =
torchvision.models.detection.fasterrcnn_resnet50_fpn(pretrained=True)
faster_rcnn_model.eval()

# Function to process and show images with FCN segmentation and
Faster-RCNN detection
def process_image(image, model_fcnn, model_frcnn):
  transform = transforms.Compose([
      transforms.ToTensor(), # Convert the image to a PyTorch tensor
  ])

  image_tensor = transform(image)

  # FCN Semantic Segmentation
  with torch.no_grad():
      output_fcn = model_fcnn(image_tensor.unsqueeze(0)) # Add batch
dimension
                                        output_fcn_predictions =
```

```
output_fcn['out'].argmax(1).squeeze().cpu().numpy()

  # Faster R-CNN Object Detection
  with torch.no_grad():
    output_frcnn = model_frcnn([image_tensor])

  # Extract boxes and labels from Fatser R-CNN output
  boxes = output_frcnn[0]['boxes'].cpu().numpy()
  labels = output_frcnn[0]['labels'].cpu().numpy()

   # Plot the original image, FCN segmentation output, and Faster R-CNN
detections
  fig, ax = plt.subplots(1, 3, figsize=(15, 5))

  # Original iamge
  ax[0].imshow(image)
  ax[0].set_title('Original Image')
  ax[0].axis('off')

  # FCN Segmentation Output
  ax[1].imshow(output_fcn_predictions)
  ax[1].set_title('FCN Segmentation Output')
  ax[1].axis('off')

  # Faster R-CNN Object Detection
  ax[2].imshow(image)
  for box in boxes:
    xmin, ymin, xmax, ymax = box
    rect = plt.Rectangle((xmin, ymin), xmax - xmin, ymax - ymin,
                         fill=False, edgecolor='red', linewidth=2)
    ax[2].add_patch(rect)
  ax[2].set_title('Faster R-CNN Object Detection')
  ax[2].axis('off')

  plt.tight_layout()
  plt.show()

# Load your images (for Jupyter Notebook
'''room_image = Image.open('road.jpg')
```

```
road1_image = Image.open('road1.jpg')
road2_image = Image.open('room.jpg')'''

# UPDATED SECTION FOR COLAB (previously images were loaded directly
using Image.open)

from google.colab import files

uploaded = files.upload()

# Automatically get correct uploaded file names
file_names = list(uploaded.keys())

room_image = Image.open(file_names[0])
road1_image = Image.open(file_names[1])
road2_image = Image.open(file_names[2])

# Process each image with the models
process_image(room_image, fcn_model, faster_rcnn_model)
process_image(road1_image, fcn_model, faster_rcnn_model)
process_image(road2_image, fcn_model, faster_rcnn_model)
```

OUTPUT:

PRACTICAL NO. 09

21-02-2026

AIM: Music Generation
Text-to-Music Generation using MusicGen.
To generate a short music clip (≈15–20 seconds) from a natural language text prompt using a pre-trained deep learning model.

CODE:

```
# A) Text-Music (≈15~20s)
# pip install -U torch torchaudio transformers accelerate

import torch, torchaudio
from transformers import AutoProcessor, MusicgenForConditionalGeneration

MODEL = "facebook/musicgen-small"
processor = AutoProcessor.from_pretrained(MODEL)
model = MusicgenForConditionalGeneration.from_pretrained(MODEL)

# prompt = "warm Lo-fi beat with soft piano and vinyl crackle"
prompt = "An Indian classical music performance with sitar and tabla,
gentle rhythm and melodic improvisation."
inputs = processor(text=[prompt], return_tensors="pt")
model.generation_config.do_sample = True
model.generation_config.guidance_scale = 3.0          # creativity vs.
prompt adherence
model.generation_config.max_new_tokens = 50*18   # ~50 tokens/sec → ~18

audio = model.generate(**inputs)                        # (1, channels,
samples)
sr = model.config.audio_encoder.sampling_rate
torchaudio.save("text_music.wav", audio[0].cpu(), sr)
print("Saved text_music.wav")


'''
cfg = model.generation_config
cfg.do_sample = True
```

```
cfg.temperature = 1.0     # < 1.0 = more conservative; > 1.0 = more
exploratory
cfg.top_k = 50          # limit sampling to top-K most likely tokens
cfg.top_p = 0.95         # nucleus sampling: smallest set with cumulative
prob 2 p
cfg.guidance_scale = 3.0
cfg.max_new_tokens = 50 * 20
'''
```

```
# Download It To Your Computer
from google.colab import files
files.download("text_music.wav")
```

OUTPUT:

text_music.wav